TrustCore

Solution Design Document

IBM Z Datathon 2026 | Track: Real-time AI for Critical Decisions

1. Problem

Fraud, fake identities, and insider threats strike in seconds, but most security systems cannot detect and stop them in time. As organizations digitize faster than ever, payment fraud is growing more sophisticated through deepfakes, synthetic identities, and AI-generated phishing; identity verification must scale without letting fake documents slip through; and insider or access threats go undetected because access patterns are too large for manual review. Most companies respond with slow, disconnected tools that cannot keep pace with real-time transaction volume, leaving small businesses, gig workers, and everyday users to bear the real cost.

2. Proposed Solution

TrustCore is a real-time AI engine that scores every transaction, document, and access event from 0 to 100 for trust, flagging risks instantly with clear, human-readable reasons. It combines a lightweight anomaly-detection model with rule-based domain logic to make each decision in a single pass, so speed and explainability are never a trade-off.

3. Target Audience

- Gig economy workers and small merchants using UPI/digital payments daily, vulnerable to fraud with no dedicated security team.
- Small businesses and startups onboarding new users or vendors at scale, needing fraud and identity checks without enterprise-level budgets.
- Internal security and compliance teams who need one unified, explainable view of transaction, identity, and access risk.
- Regulators and compliance auditors requiring proof of why an AI decision was made, relevant to DPDP Act and GDPR-style rules.

4. System Architecture

The pipeline flows in four stages: (1) Data layer, structured transaction, document, and access-log data; (2) Scoring engine, an Isolation Forest anomaly model combined with explicit rule logic to produce a hybrid trust score; (3) API layer, a lightweight REST endpoint that scores new events on demand and returns a score, flag, and explanation; (4) Dashboard, a live view of flagged events with drill-down explanations for human reviewers.

Stage	Component	Output
1. Data	Transaction / document / access-log datasets	Cleaned, feature-engineered data
2. Scoring	Isolation Forest + rule-based logic	0-100 trust score, flag, explanation
3. API	Flask REST endpoint	Real-time score-on-demand for new events

4. Dashboard	Live UI with filters	Human-reviewable flagged events

5. Datasets Used

- Simulated UPI/payment transaction data: modeled user behavior (amount, time, location, frequency) with ~5% planted fraud patterns.
- Public identity/document datasets (e.g. MIDV-500) for document authenticity and tampering detection.
- Public fraud detection datasets (e.g. Kaggle Credit Card Fraud Detection) to benchmark the anomaly-detection approach.
- Simulated access-log data with planted anomalies, for the insider-threat detection module.

No real user PII is used anywhere. All datasets are public/open-source or synthetically generated, keeping the project compliant with data privacy principles by design.

6. Why This Matters

Fraud and identity theft disproportionately hurt those with the least recourse: small vendors, gig workers, and everyday users. Compliance frameworks such as India's DPDP Act and GDPR increasingly demand systems that can prove why a decision was made, not just that it was made. TrustCore makes trust-and-safety infrastructure accessible to organizations that cannot afford enterprise-grade fraud teams, while remaining explainable, real-time, and audit-ready by design.

7. Team & Execution Plan

Role	Responsibility	Deliverable
Data & Feature Engineer	Source, clean, and structure datasets; define anomaly features	Cleaned datasets + data dictionary
ML / Anomaly Engineer	Build and tune the core scoring engine; add explainability	Working model + scoring API
Security & Trust Lead	Define threat patterns; convert to rule logic; compliance narrative	Rule logic + threat model doc
Dashboard & Presentation Lead	Build live dashboard; prepare pitch deck and demo	Working demo + pitch deck

8. Technical Stack & IBM Z Relevance

Python (pandas, scikit-learn) for data processing and the anomaly model; Flask for the real-time scoring API; a lightweight HTML/JS dashboard for visualization. This architecture reflects the kind of high-volume, low-latency transaction analysis that IBM Z and LinuxONE are purpose-built for: enterprise-scale, real-time trust decisions without compromising explainability or auditability.